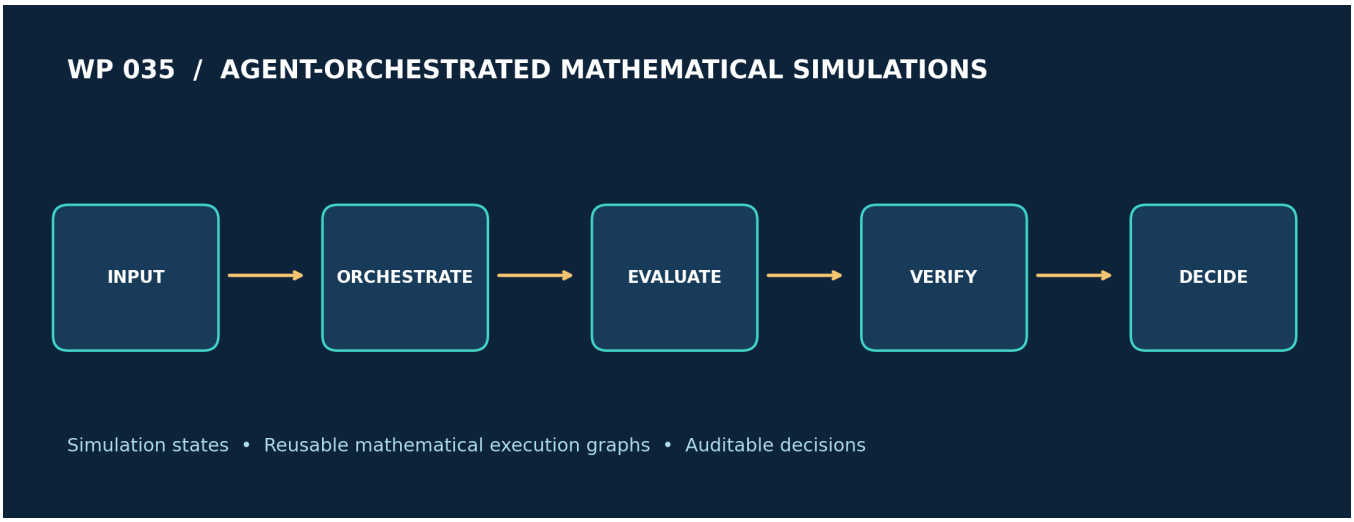
PACEVAL | SCIENTIFIC RESEARCH DEMONSTRATOR

Agent-Orchestrated Mathematical Simulations

Reusable simulation expressions, deterministic workflow decisions, recorded retries, independent validation and controlled mathematical versioning.



Executed result: 100 deterministic scenarios; 6 persistent paceval graphs; 38,562 scalar evaluations; 5,257 batch evaluations; 52 observed-triggered retries; 79 accept and 21 review decisions. Original paceval Linux SDK execution and CTest passed.

Cover visual: conceptual editorial illustration of workflow stages. It is not a measured simulation result. The study uses synthetic models, not a live AI agent or a network MCP server.

1. Industrial application and state of practice

Industrial simulation workflows repeatedly evaluate physical or mathematical models, compare simulated outputs with recorded targets, and decide whether to approve a candidate or reconfigure and rerun it. This pattern appears in digital-twin model assessment, process engineering, software-in-the-loop validation and engineering design studies. The practical need is not simply arithmetic: it includes repeatable orchestration, input contracts, provenance, result interpretation and controlled model changes.

Research context in 2026

The Model Context Protocol (MCP) formalizes typed tool interfaces and structured tool outputs. The July 28, 2026 specification highlights explicit tool schemas, versioning and security controls. Scientific workflow research increasingly examines dynamic orchestration, reproducibility, execution provenance and responsible autonomous control. These are distinct requirements from evaluating an explicit mathematical expression.

Specific contribution of WP_035

This experiment implements a deterministic offline analogue of the application layer: a scripted controller selects one of three simulation functions, invokes paceval through persistent mathematical graphs, computes an objective, optionally retries under a declared policy, and records an auditable final decision. It is a mathematical-workflow feasibility demonstration. It does not use a language model or implement MCP transport, authorization or autonomous scientific discovery.

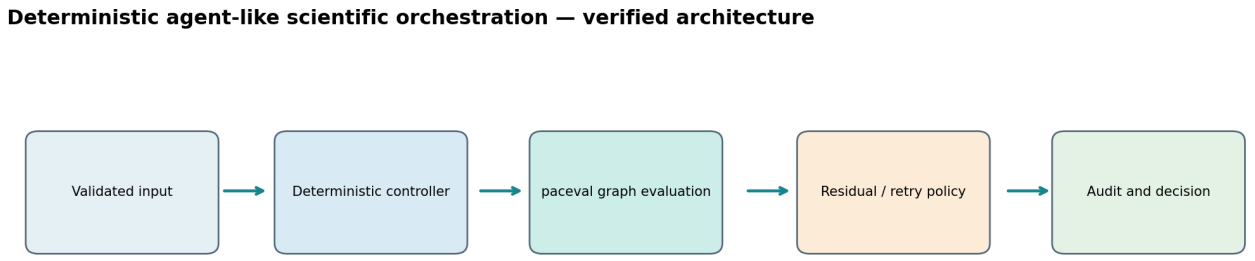


Figure 1. Demonstrated division of responsibility between scripted controller, paceval kernel and independent validation.

2. Defined mathematical simulation models

Three dimensionless and deliberately simple ordinary differential-equation families provide meaningfully different operator combinations. The host uses fixed-step explicit Euler time integration (h = 0.025), while each right-hand side is an immutable paceval expression graph.

Family A: relaxation analogue

$$dx/dt = -p\ (x - q)$$

Illustration: relaxation of a controlled process state toward its target q, with positive response coefficient p.

Family B: damped periodic forcing

$$dx/dt = -p\ x + q\ \sin(t)$$

Illustration: a forced response with damping coefficient p, input amplitude q and known synthetic forcing coordinate t.

Family C: logistic evolution

$$dx/dt = p\ x\ (1 - x/q)$$

Illustration: growth with saturation q. The controller uses q > 0 and positive initial state in the simulated study.

Residual and squared objective

$$r = \mathrm{sqrt}((x - y)^2)\ /\ (\sigma + 10^{-6}),\ J = ((x - y)/(\sigma + 10^{-6}))^2$$

The observation y is deterministic and generated from the same model family with a hidden fixed parameter. The small additive constant avoids a zero denominator. It is not a probabilistic uncertainty guarantee. The host averages J across 64 observations for its scenario decision.

3. paceval graph architecture and version control

Six computations are created using the unchanged vendor adapter files paceval_main.h and paceval_main.cpp, linked to a compatible external shared library. The scalar evaluation API is used inside every simulation time step, and the SDK batch API evaluates the residual surface and additional reference samples.

Graph identifier	Actual computation and role
1–3	Three dimensionless model RHS expressions
4	Normalized absolute residual
5	Normalized squared residual
6	Thermal model revision with a cubic term

Controlled formula revision

Original: f1 = −p(x − q). Revised: f2 = −p(x − q) − 0.02(x − q)³. Both graphs are created independently. Neither expression is silently overwritten; both remain available throughout the run.

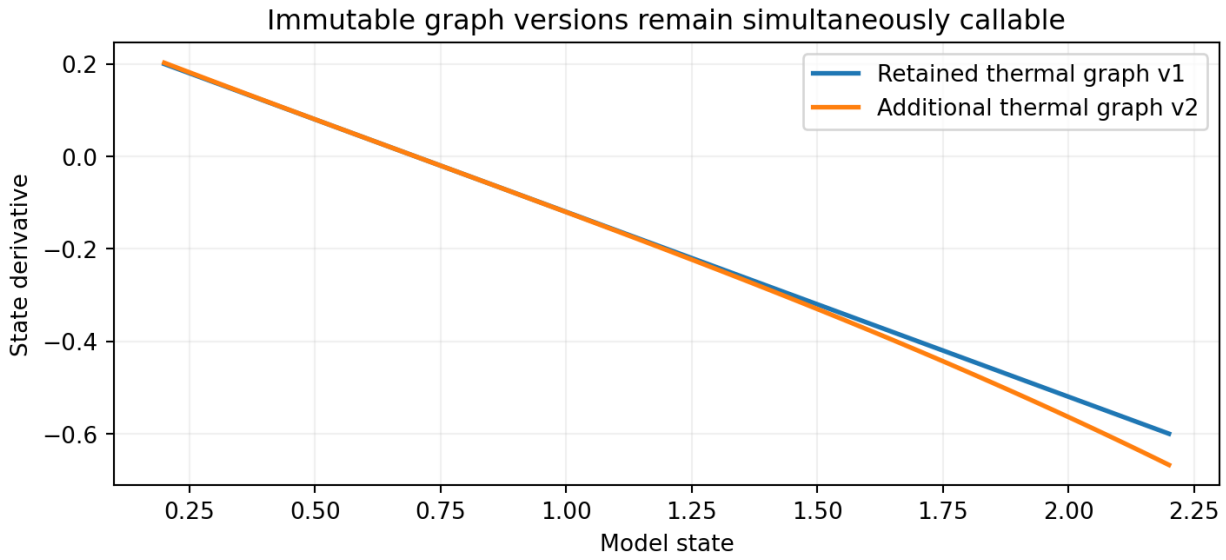


Figure 2. Retained original and revised thermal expressions evaluated with paceval. No claims are made about model superiority.

4. Deterministic workload and orchestration policy

A total of 100 scenarios selects model family via scenario identifier modulo three. Each case generates 64 clean synthetic observations with a declared parameter, deterministic starting state and step size. A candidate parameter is selected by a fixed perturbation function. The graph evaluates the candidate state and squared normalized objective.

The controller performs one additional attempt only when the first average objective exceeds 0.005. The retry parameter is obtained from the first observed numerical derivative using the known selected model equation, then averaged with the initial candidate. The controller never reads the hidden true parameter in this decision. Such one-step estimation works unusually well for these noise-free, directly identifiable demonstrations; it does not generalize to arbitrary engineering inverse problems.

A final average squared objective at or below 0.005 receives ACCEPT; all other results receive REVIEW. These statuses are synthetic workflow labels without real-world safety significance.

Recorded execution property	Observed output
Scenarios and saved step traces	100 scenarios / 6,400 step records
Initial-score-triggered retries	52
Final policy decisions	79 ACCEPT / 21 REVIEW
Immutable formula definitions	6 distinct mathematical graph handles

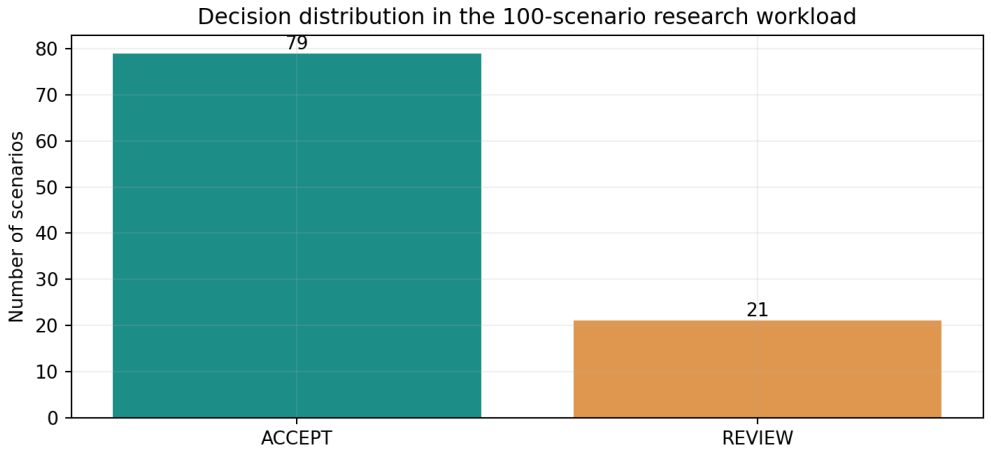


Figure 3. Workflow outcomes for the declared synthetic threshold.

5. Three simulated model families and scored behavior

Three different reusable mathematical simulation graphs

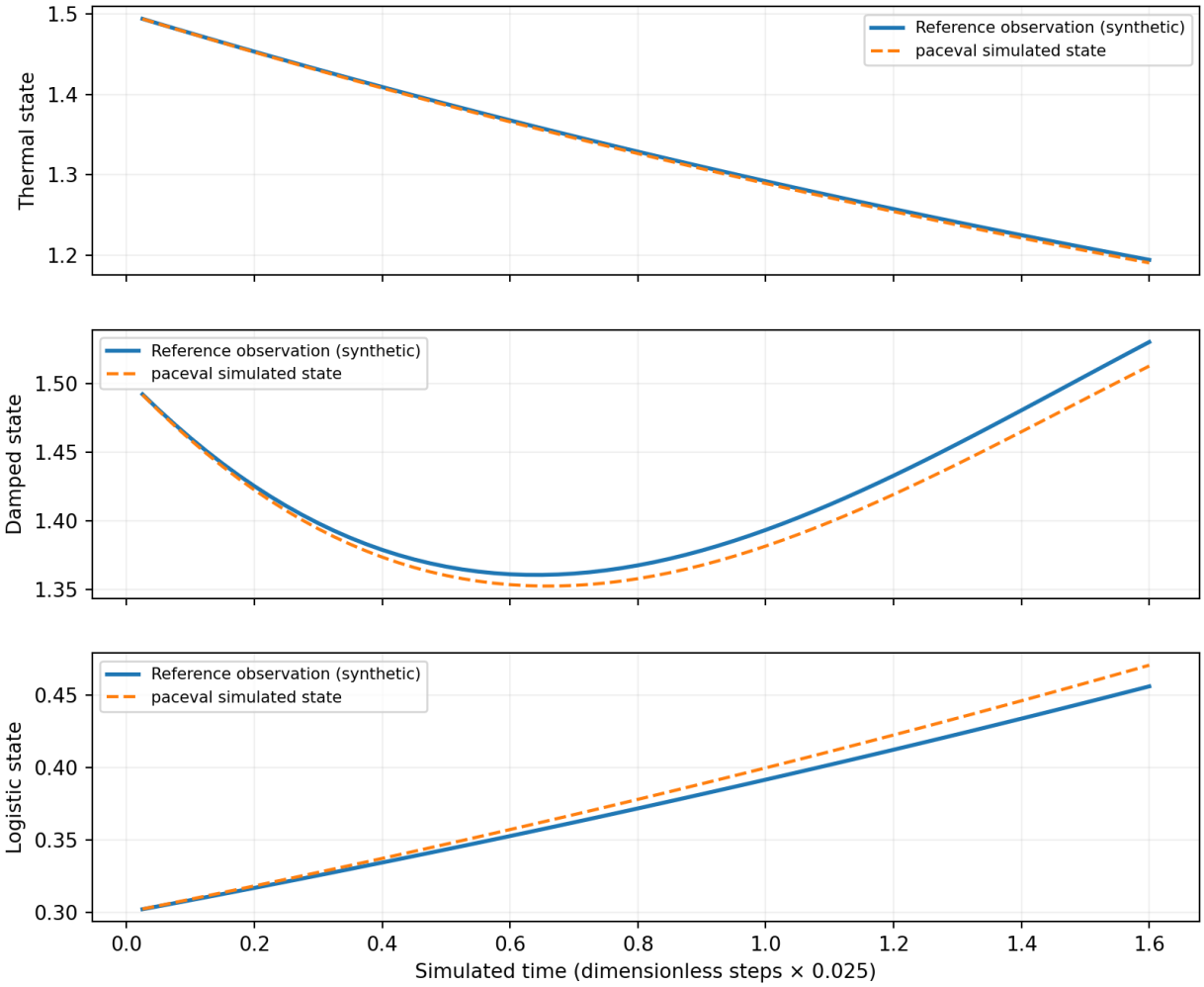


Figure 4. Synthetic observations and paceval-integrated states for one representative scenario per model family. The host owns the explicit Euler step; paceval owns the evaluated model right-hand side.

6. Retry policy and per-model differences

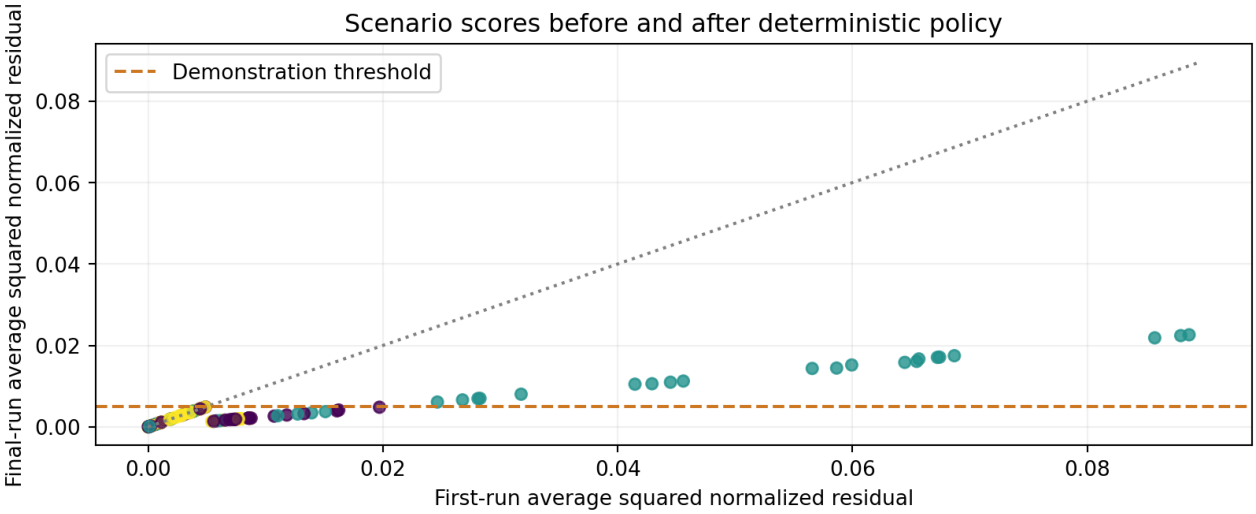


Figure 5. Scenario scores before and after the deterministic retry policy; the dashed horizontal line is the chosen demonstration acceptance threshold.

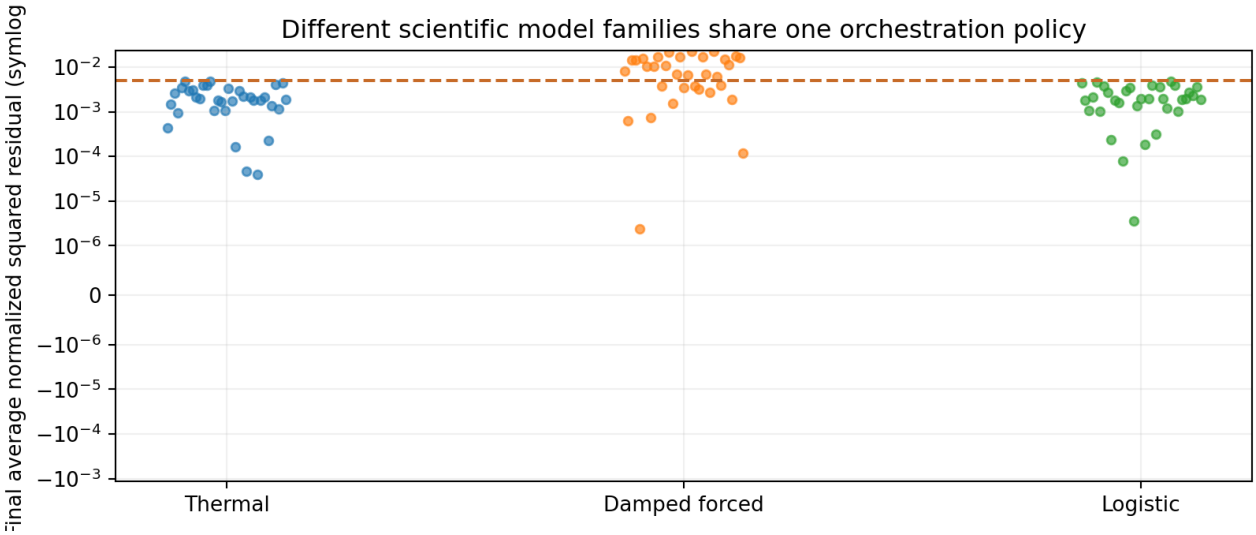


Figure 6. Distribution of final objective values by model family. Different scales and conditions can affect interpretation.

7. True numerical response surfaces produced by paceval

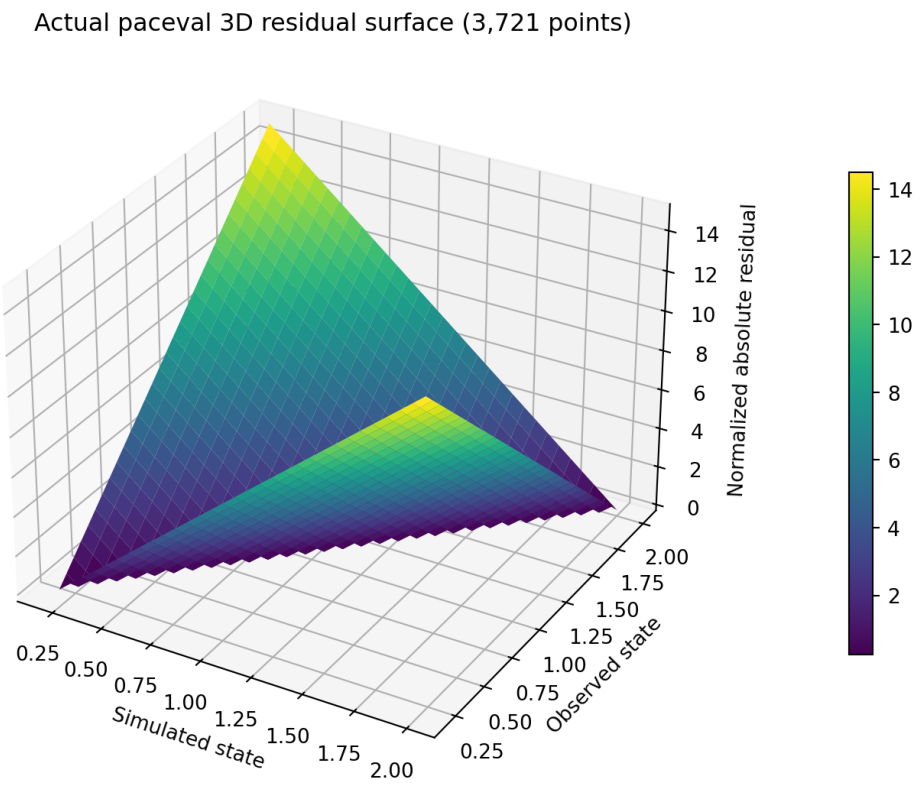


Figure 7. Three-dimensional normalized residual response surface; 3,721 locations were evaluated using the paceval batch API.

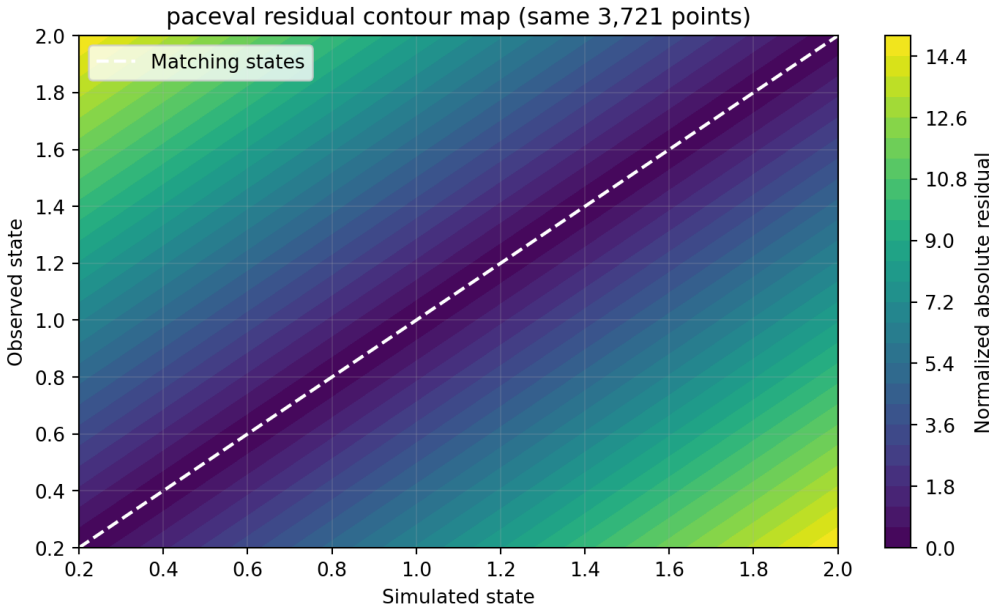


Figure 8. Two-dimensional contours from the same output grid.

8. Independent scientific audit and reproduction

The actual Linux CMake build and SDK integration passed CTest. The C++ program independently checks every evaluated right-hand-side output and derived score against separately implemented numerical expressions. Its audit file records 38,562 scalar evaluations, 5,257 batch evaluations and 43,657 internal numerical reference checks. No unpublished speed or precision advantage is claimed.

A separate Python program evaluates 449 selected exported values with 80-digit mpmath reference arithmetic and passes its configured tolerance. Dataset verification checks the count of 100 scenarios, 6,400 trace values, 3,721 residual grid points, unique formula registry, final policy status logic and retry logs. All published numerical plots are regenerated from the exact CSV outputs.

Build and reproduce (from repository root)

```
cmake -S src -B build -DCMAKE_BUILD_TYPE=Release -DPACEVAL_RUNTIME=/absolute/path/to/compatible/libpaceval_linux_sharedLIB.so

cmake --build build -j
ctest --test-dir build --output-on-failure
./build/agent_simulations --out examples
python3 src/verify_results.py
python3 src/high_precision_reference.py
python3 src/make_figures.py
python3 src/check_readme_math.py
```

Evidence and scientific limitations

All observations are synthetic, clean, dimensionless and produced by the frozen reference model. Fixed-step integration has discretization error, and the scenario model is not calibrated against measured physical data. The retry estimator has access to the first observation and a known analytical model but not to the hidden generating parameter. Finite numerical validation cannot establish global error guarantees or safe decisions in untested operating regions.

The original paceval runtime binary is external and intentionally not distributed in the ZIP. Its upstream redistribution conditions must be checked independently.

9. Industrial interpretation, future research and sources

The central scientific result is that mathematical execution graphs from paceval can be combined into a reproducible, policy-directed simulation workflow without reconstructing the underlying expressions for each scenario. Separate result logs preserve model choice, parameter revision, mathematical residual and final decision. This is an appropriate prototype for the calculation layer of an AI-agent tool system.

Next industrial step

A follow-on study should integrate this calculation layer into a controlled MCP tool service with input/output schemas, authorization, approval for sensitive operations, resource quotas, provenance, model version hashes, and real external simulator or observational datasets. Only then could a real AI-agent orchestrator be tested for planning quality, error recovery, security and operational reliability. Closed-loop simulations should be checked for convergence, parameter identifiability and observation noise under realistic conditions.

Selected external references

Model Context Protocol, Tools specification 2026-07-28. https://github.com/modelcontextprotocol/modelcontextprotocol/blob/main/docs/specification/2026-07-28/server/tools.mdx

Model Context Protocol, official 2026-07-28 release announcement. https://blog.modelcontextprotocol.io/posts/2026-07-28/

Alnasir (2026), AI-Driven Scientific Computing Workflows: A Systems Review of Orchestration, Execution, Reproducibility and Provenance, preprint. https://arxiv.org/abs/2609.21162

NIST AI Risk Management Framework. https://www.nist.gov/itl/ai-risk-management-framework

paceval Mathematical Engine, official source and API. https://github.com/paceval/paceval

Conclusion

The specified deterministic orchestration, six persistent mathematical graphs and audited synthetic scientific calculations provide a positive feasibility result for reusable paceval computation within an agent-like simulation workflow. They do not establish a live LLM-agent system, MCP conformance, industrial simulation validity or a safety certification.